PROJECT SYNOPSIS

"Cloud based ERP management system"

Master of Computer Applications

Submitted By

Sakshi B Sankpal 2GI24MC085

Under the Guidance of

Prof. N.N.Anvekar

Department of Master of Computer Applications

Karnataka Law Society's

GOGTE INSTITUTE OF TECHNOLOGY

Udyambag, Belagavi -590008

Introduction

Enterprise Resource Planning (ERP) systems are used by organizations to manage internal processes such as employee management, task assignment, attendance tracking, and communication between administrators and users. ERP systems help organizations integrate different activities into a single system, which improves efficiency and makes management easier.

In many organizations, employee records, attendance, and task tracking are managed using manual methods such as spreadsheets or separate software tools. This approach can lead to data duplication, errors, and difficulty in tracking information. An ERP system provides a centralized platform where all information can be stored and accessed in a structured way. This project implements a DevOps-based ERP management system that uses modern cloud technologies and automation tools. The system allows administrators to manage users, assign tasks, track attendance, review work submissions, and communicate with interns through announcements or messages.

Interns can register and log in to the system, view assigned tasks, submit their work, mark daily attendance, request leave, and receive notifications from the administrator. The system also includes certificate generation features that help manage internship completion certificates.
The application is developed using Flask for the backend and React for the frontend, while SQLite is used as the database to store user data, tasks, attendance records, and other system information.

To improve deployment and system management, DevOps practices are implemented in this project. DevOps focuses on automation, collaboration, and continuous monitoring to improve the software development lifecycle.
DevOps tools such as Docker and Docker Compose are used for containerization and managing application services. Git and GitHub are used for version control and collaboration. GitHub Actions is used to implement CI/CD pipelines that automate the build and deployment process.

The system is deployed on AWS EC2 cloud infrastructure, which provides scalable and reliable hosting for the application**.** Nginx is used as a reverse proxy to handle user requests and improve system performance.
To monitor system performance and resource usage, Prometheus is used to collect metrics and Grafana is used to visualize monitoring data through dashboards. These tools help administrators track system health and detect issues quickly.

The objective of this project is to demonstrate how DevOps practices can improve software development, automate deployment processes, enhance system monitoring, and provide scalable infrastructure for modern web applications.

Functional Requirements

User Management

- User registration and login
- Role-based access (Admin and Intern)
- Profile management

Admin Features

- Approve or reject intern registrations
- Assign tasks
- Monitor intern attendance
- Review submissions and provide feedback
- Send announcements and messages

Intern Features

- Login to the system
- View assigned tasks
- Submit task work
- Mark daily attendance
- Request leave
- Receive notifications

Certificate Management

- Generate internship completion certificates
- Verify certificates

Non-Functional Requirements

Performance

- The system should support multiple users simultaneously.

Scalability

- Container-based architecture allows scaling when required.

Security

- Authentication and authorization mechanisms protect system access.

Reliability

- Monitoring ensures system stability and uptime.

Maintainability

- CI/CD pipelines automate build and deployment.

Hardware Requirements

- Processor: Intel i5 or equivalent
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB free disk space
- Internet: Stable internet connection

Software Requirement

The following software tools and technologies are used to develop, deploy, and monitor the ERP system using DevOps practices.

Programming Language

Python

Node.js

Backend Framework

Flask

Frontend Framework

React

Database

SQLite

Containerization

Docker

Container Orchestration

Docker Compose

Version Control

Git

GitHub

CI/CD Automation

GitHub Actions

Cloud Platform

AWS EC2

Reverse Proxy

Nginx

Monitoring

Prometheus

Visualization

Grafana

These technologies help in building a scalable ERP system and implementing DevOps practices such as containerization, automated deployment, and monitoring.

DevOps Responsibilities

In this project, the DevOps intern is responsible for implementing automation, deployment, and monitoring processes for the ERP system.

Key responsibilities include:

- Managing source code using Git and GitHub
- Creating Docker containers for application services
- Managing multi-container environment using Docker Compose
- Implementing CI/CD pipeline using GitHub Actions
- Deploying the application on AWS EC2 cloud server
- Configuring Nginx as reverse proxy
- Monitoring system performance using Prometheus and Grafana

System Architecture

User

↓

Nginx Reverse Proxy

↓

React Frontend

↓

Flask Backend

↓

SQLite Database

Literature Review

Modern DevOps practices integrate development and operations to improve the software lifecycle. Containerization technologies such as Docker allow applications to run in isolated environments, ensuring consistency across development and production.

Cloud platforms such as AWS provide scalable infrastructure for deploying applications. Monitoring tools like Prometheus and Grafana enable real-time performance monitoring.

The integration of these technologies forms a DevOps pipeline that supports automation, scalability, and reliable system deployment.

Benefits of DevOps Implementation

- Automated deployment
- Faster development cycles
- Improved reliability
- Continuous monitoring
- Simplified infrastructure management
- Scalability

Conclusion

This project demonstrates how DevOps practices improve the deployment and management of ERP systems. By integrating containerization, CI/CD pipelines, cloud infrastructure, and monitoring tools, the system becomes scalable, reliable, and efficient.